# Foodmaster  Chatbot Report

Foodmaster is a chatbot focused on focused on food, recipes and cooking instructions. The bot uses regular expressions and stored intent patterns.  I made it food focused so the conversation has a clear purpose.but general topic intents and fallback response move beyond food when needed.

The main use case is viewing a recipe menu, finding  a dish, asking for ingredients or the cooking steps, requesting a meal suggestion or get some short food facts. The current recipe data contains six dishes-  Chicken biriyani, Shakshuka, Banana Pancakes, Vegetable fried rice, Pasta Arrabbiata and Lentil Dahl. The food fact file also contains six facts  -  Umami, Fermantation, Al dente , Roux, Tempering Spices, Mise en place

The chatbot was designed in a way that it can handle follow up message. You can ask for the menu, mention  "chicken biriyani", then say "yes please, can i have the ingredients?" and  ask "what about the cooking methods?" The chatbot will store the last recipe and use it to interpret the shorter follow ups. It can also remember a name, food preference and vegetarian or vegan choice during the session you are in.

The intents.json file organises the chatbot into 30 intent categories like greetings, menu requests, recipes, ingredients, food facts, recommendations, preferences, sentiment, topic changes and general help. All intent contain a tag, a regular expression pattern and one or more response templates. Some intents also contains phrase lists for recognising different ways of asking the same questions (not all of them).

clean_text() will standardise text by changing it to lowercase, remove punctuation and reduce repeated space. has_phrase() will check for phrase inside a longer message using escape text and word boundary. Other patterns are searched with re.search(), this mean the chatbot can recognise a keyword or phrase without requiring the entire user message to match exactly.

Named groups will capture details from a message. Like, a preference pattern keeps the text after phrases like "I prefer" into a group named preference.match.groupdict() make the value available to the response handler and .format() inserts them into a template. random.choice() selects between alternative replies where an intent has more than one

response. Recipe title are checked against the recipe data so shorted word like "biriyani" can refer to chicken biriyani

Foodmaster loads info from seperate files instead of keeping all content inside the python functions. This makes the chatbot easier to update  like recipe details, facts, example messages and intent patterns can be edited without rewriting the main conversation loop.

Intents.json - Stores intent tags, regex patterns, phrase variations and response templates

Recipes.json-  Stores recipe titles, aliases, ingredients, steps, cuising, meal types and diet

Food_facts.txt - stores short facts

Foodmaster.ipnyb - contains the chatbot code, file loading cell, scripted demo, nlp example and live chat cell

The program uses python lists and dictionaries to organise the loaded information.  recipe records are dictionaries held in a list; the session memory is another dictionary. The loader functions load*intents(), load*recipes(),load*facts(),load*script() keep file handling seperate from the message handling function.

The general topic feature is limited. Foodmaster will reply to a topic lets say "music" and reply to discuss , then recognise a request to return to food. Keeps the chatbot open to topic change without pretending to have capabilities it does not have.

Conversation memory helps with follow up questions by the keeping the last recipe and a pending action. For example after ingredients are shown a short reply such as "yes please" will trigger the offered cooking steps. Recipe suggestions also use the remembered meal type, dietary choice and food preference. Topic memory records the currenrt and previous topic so you can return to cooking.

This chatbot uses NLTK VADER to classify sentiment as positive, negative or uncertain. Notebook has week 5 text processing techniques: tokenisation, wordnet lemmatisation, porter stemming, stopword removal. They are shown as nlp exercise in the notebook, the main intent matching still reliies on the project regex patterns and phrase lists

# Testing

Testing check both repeatable script  and messahe typed in the live notebook chat. Below is a testing table

| Test | Input | Expected result | Result |
|---|---|---|---|
| Menu matching | "Could you show me the menu please" | Display recipe list | Succesfull |
| Recipe alias and memory | "I crave chicken biriyani" | Recognise the alias for chicken biriyani and store dish for follow up | Succesfull |
| Ingredient follow up | "Yes, please can i have the ingredients" | Show ingredient for previosuly selected dish | Succesfull |
| Cooking method follow up | "What about the cooking method" | Show the step for remembered dish | Succesfull |
| Food fact | What is umami | Show the umami fact from foodfacts | Succesfull |
| Memory and topic switch | "I am vegetarian"<br>"What do you remember"<br>"back to cooking" | Remember your choice and responds to return to food request | Sucessfull |
| Sentiment | Postive and Negative sentences-<br>I'm feeling really disappointed today.<br><br>I'm feeling happy toay. | Show sentiment labels | Sucessfull |